BTP610 Test 3: W8 Maps | W9 Location | W10 Firebase

W8 Imports + Setup

```
import { useRef, useState } from "react";
import { Text, TouchableOpacity,
  View } from "react-native";
import MapView, { Marker, Region }
  from "react-native-maps";

npm install react-native-maps

const region: Region = {
  latitude: 43.64272,
  longitude: -79.38705,
  latitudeDelta: 0.0045,
  longitudeDelta: 0.0065,
};

const mapRef = useRef<MapView>(null);
```

W8 Map + Marker

```
type Place = { name: string; lat: number;
  lng: number; };

const [markerList, setMarkerList] =
  useState<Place[]>([]);

<MapView ref={mapRef}
  initialRegion={region}
  style={{ flex: 1 }}>
  {markerList.map((item, index) => (
    <Marker key={index}
      coordinate={{
        latitude: item.lat,
        longitude: item.lng,
      }}
      title={item.name}
    />
  ))}
</MapView>
```

W8 Buttons

```
const addMarker = () => {
  const pos = Math.floor(
    Math.random() * places.length
  );
  const selected = places[pos];
  setMarkerList([...markerList, selected]);
  mapRef.current?.animateToRegion({
    latitude: selected.lat,
    longitude: selected.lng,
    latitudeDelta: 1,
    longitudeDelta: 1,
  });
};

const clearMarkers = () => {
  setMarkerList([]);
  mapRef.current?.animateToRegion(region);
};

<TouchableOpacity onPress={addMarker}>
  <Text>Add Marker</Text>
</TouchableOpacity>
<TouchableOpacity onPress={clearMarkers}>
  <Text>Clear</Text>
</TouchableOpacity>
```

W8 Reminders

```
MapView = map
Marker = pin
initialRegion = start only
region = controlled map
smaller delta = more zoom
coords use latitude/longitude
useRef<MapView>(null)
animateToRegion(...)
```

W9 Imports + State

```
import * as Location from
  "expo-location";
import { Alert, TextInput } from
  "react-native";

npx expo install expo-location
npx expo install react-native-maps

const [region, setRegion] =
  useState<Region | null>(null);
const [address, setAddress] =
  useState("");
const [currentAddress,
  setCurrentAddress] = useState("");
```

W9 Permission + GPS

```
const { status } = await
  Location.requestForegroundPermissionsAsync();
if (status !== "granted") {
  Alert.alert("Permission Denied");
  return;
}

const location = await
  Location.getCurrentPositionAsync({
    accuracy: Location.Accuracy.Highest,
  });

const newRegion: Region = {
  latitude: location.coords.latitude,
  longitude: location.coords.longitude,
  latitudeDelta: 0.0045,
  longitudeDelta: 0.0065,
};
setRegion(newRegion);
```

W9 Reverse + Forward

```
const reverseGeocode = async (
  lat: number, lng: number
) => {
  const result = await
    Location.reverseGeocodeAsync({
      latitude: lat,
      longitude: lng,
    });
  if (result.length > 0) {
    setCurrentAddress(
      result[0].formattedAddress + ""
    );
  }
};

const forwardGeocoding = async () => {
  const result = await
    Location.geocodeAsync(address);
  if (result.length === 0) return;
  setRegion({
    latitude: result[0].latitude,
    longitude: result[0].longitude,
    latitudeDelta: 0.0045,
    longitudeDelta: 0.0065,
  });
};
```

W9 Full Flow

```
const getCurrentLocation = async () => {
  const { status } = await
    Location.requestForegroundPermissionsAsync();
  if (status !== "granted") return;

  const location = await
    Location.getCurrentPositionAsync({
      accuracy:
        Location.Accuracy.Highest,
    });

  const newRegion: Region = {
    latitude: location.coords.latitude,
    longitude: location.coords.longitude,
    latitudeDelta: 0.0045,
    longitudeDelta: 0.0065,
  };

  setRegion(newRegion);
  reverseGeocode(
    location.coords.latitude,
    location.coords.longitude
  );
};
```

W9 Input + Map

```
<TextInput value={address}
  placeholder="Enter Address"
  onChangeText={setAddress}
/>
<TouchableOpacity
  onPress={forwardGeocoding}>
  <Text>Search</Text>
</TouchableOpacity>

{region && (
  <MapView style={styles.map}
    region={region}>
    <Marker coordinate={{
      latitude: region.latitude,
      longitude: region.longitude,
    }}
    title="Selected Location"
    description={currentAddress} />
  </MapView>
)}
```

W9 Reminders

```
geocodeAsync(address)
  address -> coords
reverseGeocodeAsync({...})
  coords -> address
requestForegroundPermissionsAsync()
getCurrentPositionAsync()
check result.length
```

W10 Imports + Setup

```
import { initializeApp }
  from "firebase/app";
import { getAuth,
  onAuthStateChanged,
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut, User }
  from "firebase/auth";
import { getFirestore,
  collection, addDoc, getDocs,
  doc, updateDoc, deleteDoc }
  from "firebase/firestore";

npm install firebase

const app = initializeApp(firebaseConfig);
const firebaseAuth = getAuth(app);
const fireDB = getFirestore(app);
```

W10 Auth

```
const signUp = async () => {
  await createUserWithEmailAndPassword(
    firebaseAuth, email, password
  );
};

const signIn = async () => {
  await signInWithEmailAndPassword(
    firebaseAuth, email, password
  );
};

const logout = async () => {
  await signOut(firebaseAuth);
};
```

W10 Listener + Guard

```
const [user, setUser] =
  useState<User | null>(null);

useEffect(() => {
  const unsubscribe =
    onAuthStateChanged(
      firebaseAuth,
      (user) => setUser(user)
    );
  return unsubscribe;
}, []);

if (!user) {
  return <Redirect href="/signIn" />;
}
return <Redirect href="/BookList" />;
```

W10 Firestore

```
await addDoc(collection(fireDB, "books"), {
  title: "Atomic Habits",
  author: "James Clear",
});

const snap = await getDocs(
  collection(fireDB, "books")
);
const books = snap.docs.map((doc) => ({
  id: doc.id, ...doc.data(),
}));

await updateDoc(doc(fireDB, "books", id), {
  title: "New Title",
});

await deleteDoc(doc(fireDB, "books", id));
```

W10 Reminders

```
initializeApp = start Firebase
getAuth = auth service
getFirestore = DB
onAuthStateChanged = watch auth
Firestore = NoSQL cloud DB
Auth = sign up / sign in / sign out
```